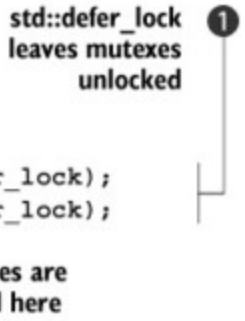
```
class some_big_object;
void swap(some_big_object& lhs,some_big_object& rhs);
class X
{
private:
    some_big_object some_detail;
    std::mutex m;
public:
    X(some_big_object const& sd):some_detail(sd){}

    friend void swap(X& lhs, X& rhs)
    {
        if(&lhs==&rhs)
            return;
        std::unique_lock<std::mutex> lock_a(lhs.m,std::defer_lock);
        std::unique_lock<std::mutex> lock_b(rhs.m,std::defer_lock);
        std::lock(lock_a,lock_b);
        swap(lhs.some_detail,rhs.some_detail);
    }
};
```



In listing 3.9, the `std::unique_lock` objects could be passed to `std::lock()` ❷ because `std::unique_lock` provides `lock()`, `try_lock()`, and `unlock()` member functions. These forward to the member functions of the same name on the underlying mutex to do the actual work and just update a flag inside the `std::unique_lock` instance to indicate whether the mutex is currently owned by that instance. This flag is necessary in order to ensure that `unlock()` is called correctly in the destructor. If the instance *does* own the mutex, the destructor *must* call `unlock()`, and if the instance *does not* own the mutex, it *must not* call `unlock()`. This flag can be queried by calling the `owns_lock()` member function.

As you might expect, this flag has to be stored somewhere. Therefore, the size of a `std::unique_lock` object is typically larger than that of a `std::lock_guard` object, and there's also a slight performance penalty when using `std::unique_lock` over `std::lock_guard` because the flag has to be updated or checked, as appropriate. If `std::lock_guard` is sufficient for your needs, I'd therefore recommend using it in preference. That said, there are cases where `std::unique_lock` is a better fit for the task at hand, because you need to make use of the additional flexibility. One example is deferred locking,